Behavioral (Ownership)

Prem

01 // HOW THIS ROUND IS SCORED

Ownership — What Meta Scores

4–5 questions in 30 min. Each story ≤5 min, aim 3:00–4:30. Going over costs volume. Rehearse aloud with timer.

VISIBLE KEY — OWNERSHIP

I, not we

Lead with your actions. Name what you designed, wrote, drove. Trade-off = yours.

TIMING RULE

STAR 30·30·120·30s

Situation ≤30s, Task ≤30s, Action ≤2m, Result ≤30s. No filler.

3:30 target

RESULT FORMAT

%, $, hrs, ms

End measurable. Then what you'd change — growth signal.

Before → After

PRACTICE

Timer Drill

Silent prep hides mumbles. Record, listen for “we” vs “I”.

Day before: 4 back-to-back

≠ **No heroes vs villains**

Frame as trade-offs, not personalities. Learning > blame. “I would've raised earlier” lands.

! **Follow-ups are scored**

“What if manager said no?” — have partner drill. Ownership lives in follow-ups, not opener.

STORY 1 — OWNERSHIP · END-TO-END

Led Project End-to-End

Prompt: Tell me about a time you led a project end-to-end.
Use when: ownership, delivery, proudest project. Real: data-quality reconciliation.

MEASURABLE OUTCOME

~~12%~~ → **0.3%**

Finance trust restored. Retired old report 60 days later. Row-hash job <10 min on 2B rows.

Job runtime (2B rows)	**<10 min**
Alert threshold	**0.5% only**
Shadow mode	**2 weeks**
Stakeholders	**3 teams + Finance**

Scoped + Owned · Cross-functional · Row-hash trade-off · Reflective

S

SITUATION ≤30S

Analytics dashboards 12% gap vs OLTP, finance losing trust. 3 product teams depend on table. Lead DE on recon project, 4-month span.

T

TASK ≤30S

Owned daily recon job end-to-end — schema, diff logic, alert routing, rollout. Done = gap <0.5% + finance sign-off.

A

ACTION ≤2M — I, NOT WE

I designed row-hash vs full diff (2B rows <10 min). **I wrote** alert only >0.5%, not transient blips. **I drove** rollout: shadow 2 weeks, weekly finance report, single approval gate before decommission old report.

R

RESULT ≤30S + CHANGE

Gap 12% → 0.3%, finance regained trust, retired old report 60 days later. What I'd change: push back on 2-week shadow — would've caught bug a sprint earlier.

STORY 2 — DISAGREED & RESOLVED · FRICTION

Disagreed on Big-Bang Cutover

Prompt: Tell me about a time you disagreed with your manager.
Use when: conflict, pushback, influence without authority. Real: Redshift → Snowflake, ~150 tables, hard contract deadline.

S SITUATION ≤30S

Manager wanted single big-bang before quarter end. **~150 tables, 40 dbt models, 30 dashboards all dark if 1 breaks.** No quick rollback.

T TASK ≤30S

I owned cutover — validation, dbt port, dashboard rewiring. If big-bang fails, I'm on rollback at 2am.

A ACTION ≤2M — 1-PAGER + MIDDLE PATH

Didn't say "no." **I profiled 12 highest-risk tables** + 8-hour freeze cost vs 3-hr windows. Built strangler-fig plan: dual-write both warehouses, dual-materialized dbt, dashboards team-by-team, lowest-stakes first. Booked 30-min with manager + skip, acknowledged deadline, proposed 6-week hard cap + per-wave rollback (config flip).

R RESULT ≤30S + CHANGE

5 weeks, 3 waves, zero downtime. First 2 waves surfaced 6 issues that would've been catastrophic under big-bang. Hit contract +1 week spare. Change: raise day proposal landed, not week before kickoff.

1-PAGER I BROUGHT — HOMEWORK, NOT OPINION

Blast radius counted	**150 / 40 / 30**
12 highest-risk tables	**Rows + models + users**
Window: big-bang vs phased	**8h vs 3h**
Hard cap proposed	**6 weeks, no extensions**

5 wks	3	0
FULL MIGRATION	WAVES	DOWNTIME

What This Proves — Visible Keys

- Disagreed with evidence + concrete alternative
- Respected hard constraint, proposed cap
- Per-wave rollback = config change, not fire drill
- Framing: "We can do this with less risk and still hit date"

Reflection line: "Same pushback a week earlier saves a week of rushed work. I was nervous to push — burned a week."

STORY 3 — PROCESS WIN · FRAMEWORK

Built Reusable Ingestion Framework

Prompt: Tell me about a process you improved.
Use when: how you make team better, platform work adopted.
Real: 30 one-off pipelines → config-driven framework.

TIME TO NEW SOURCE

2 wk → 2 d

90% reduction. YAML config only: source, schedule, contract, target.

ON-CALL & QUALITY

70% ↓ pages

3 schema drifts caught auto. Previously silent loss.

ADOPTION IN 3 MONTHS

8 / 30 migrated

3 teams adopted in first month, pull not push. Dogfooded 2 highest-traffic.

S SITUATION ≤30S

30 one-off pipelines, 2 weeks each. Custom Airflow DAG, schema logic, tests. Half no tests, ⅓ silently dropped rows on drift. On-call hours/week chasing.

T TASK ≤30S

Only eng on framework. Users = downstream teams. Own single abstraction teams can point at source & ship in days.

A ACTION ≤2M — DIAGNOSIS → FIX → ROLLOUT

Catalogued 3 patterns (REST API, relational DB, SFTP = ~90%). Built config-driven: YAML → idempotent S3 Parquet, schema-drift detection, Great Expectations, dbt scaffolding, retries/backoff, PagerDuty+Slack. Built on Airflow+dbt, no new learning. **I dogfooded orders + customers feeds myself** before asking anyone. Wrote 1-page README + worked example + 30-min lunch-and-learn with numbers.

R RESULT + WHAT I'D CHANGE ≤30S

2 weeks → 2 days, 8 pipelines in 3 months, on-call ↓70%, 3 drifts auto-caught. What I'd change: instrument row-count + freshness day 1 — added month 2, had 2 quiet stale-data incidents gap broke dashboards.

STORY 4 — LEARNING AGILITY · RAMP-UP SPEED

Learned New System & Delivered

Prompt: Learn new tool quickly & deliver
Tool: dbt + Snowflake from scratch, 6 weeks, revenue dataset for quarter sign-off
Scope discipline: 4 models, not 10. Tests day 1.

3d	**Docs + rebuild**	1h	**Daily office hrs · 2 wks**
Rebuilt existing pipeline as learning exercise		With analytics eng for first 2 weeks	
4→	**Scoping 4 not 10**	✓	**On-time + Template**
Models + tests day 1, paired finance every 2d		Finance sign-off, became template for next 2	

S — SITUATION ≤30S

Team adopted **dbt + Snowflake 6 weeks before finance close.** Never used either. Deliverable was reconciled revenue dataset quarter sign-off.

T — TASK ≤30S

I owned dbt models + Snowflake ingestion — whole pipeline, hard deadline. Done = finance signs.

A — ACTION ≤2M — RAMP + SCOPE

I spent 3 days reading docs, rebuilt one existing pipeline. **I booked daily office hrs 2 weeks** with analytics eng. **I scoped aggressively:** 4 models not 10, tests every model day 1 because didn't trust own output. **I paired 1h /2d with finance** to validate definitions, not vacuum.

R — RESULT + CHANGE ≤30S

Delivered on time, finance sign-off, became template for next 2 pipelines. What I'd change: 30-min kickoff doc upfront — spent day re-litigating definitions that quick convo would've resolved.

STORY 5 — LEARNING AGILITY · GO REWRITE

Learned Go + Rebuilt Databricks Integration

Prompt: Learn new tool quickly & deliver
Use when: polyglot, backend service, cost reduction. Real: Python tangled scripts → Go service, 4-week hard deadline.

MEMORY
1/10th
vs Python service — static binary

COST
−40%
job-run cost via conn reuse + cancel zombies

LATENCY P95
3s → 300ms
Shared http.Client, no TLS handshake per poll

ADOPTION
3 teams / 1mo
Default API to trigger Databricks

S SITUATION — ≤30S

Python service triggering Databricks jobs: slow, expensive, tangle of scripts, no owner. **Never written Go before** — Python+Java only.

T TASK — ≤30S

Volunteered: rewrite in Go — tiny service, clean REST API, thin SDK wrapper. Production-ready 4wks, zero-downtime replace, cut cost.

A ACTION — ≤2M — 20% OF GO THAT SHIPS

Week1: Tour of Go + Databricks REST docs. Picked 4 endpoints only (`run-now`, `get`, `list`, `cancel`), built CLI with curl first. **Week2:** slice submit→poll→return using `net/http`, `context.Context` everywhere for cancel, thin client with retries + backoff + shared `http.Client`. 2×1h pairing with senior Go eng saved week. Scope: no auth refactor, no multi-cloud — drop-in + `/healthz`. **Week3:** JSON logs + Prometheus metrics. **Week4:** shadow-mode behind Python, then cutover.

R RESULT — ≤30S + CHANGE

Shipped on time, 1/10th mem, −40% cost, p95 3s→300ms, 3 teams adopted in month. Change: shadow-mode week2 not week4 — burned weekend on polling bug caught day3 in integration.

HOW I LEARNED GO FAST — VISIBLE KEYS

1. **20% that ships:** structs, interfaces, goroutines, channels, context, net/http. CLI before service.
2. **Stdlib first:** no heavy framework = fewer surprises. Context cancel kills zombie jobs.
3. **Senior leverage:** 2×1h reviews on concurrency + error handling.
4. **Instrument day 1:** metrics show where money goes. Shadow early catches what unit tests don't.

WHY GO FOR THIS?

Single static binary, great stdlib for HTTP, low mem, fast startup — perfect control-plane service orchestrating Databricks. Buy-in via shadow-mode numbers, not rewrite pitch.

ALTERNATE STORIES — WHEN YOU NEED SCALE

Unified Platform at Lazada · 5 → 1

Use when platform-lead / 10+ engineers.

Same STAR but scale proof. Small-scope = Story 1 recon (12%→0.3%). Large-scope = this variant.

STORY 6 — LED END-TO-END AT SCALE

Real-time + Campaign Data

Schema registry · Producer contracts · Streaming-first

Campaign freshness	**24h → 60s**
Last-mile breach	**6h → <2m**
Attribution	**1w → 1h query**
5 stacks	**→ 1 in 4 mo**

S — SCATTERED

5 stacks (orders, inventory, campaigns, last-mile, marketing). Campaign 24h delay, last-mile 6h+, join = 1 week ad-hoc.

T — TECH LEAD

Owned arch, governance, migration across 5, rollout 10+ eng. Done = one ingestion, contracts write-time, single API.

A — ENFORCEMENT TRADE-OFF

Central registry + **reject at write-time** with 1-week grace. RFC + 3 incidents cost. Streaming Kafka for real-time, batch for finance. Sat week each with hardest teams.

R + CHANGE

60s campaign, <2m SLA, 5→1, 10+ migrated. Change: shadow registry month earlier — 2 producers built during design retrofit.

PHASED — 3 PHASES 4 MONTHS

- **Phase 1** Registry + contracts shadow — observability only.
- **Phase 2** Last-mile + campaign live, dual-writes for rollback.
- **Phase 3** Orders, inventory, marketing, deprecate 5 legacy.

STORY 7 & 8 — SAME FRAMEWORK, ALTERNATE SAMPLES

Story 7 = Disagreement (hard deadline)

Key: "Which deadline matters — contract or cutover?" Phased still hits contract team-by-team. Rollback = config flip.

Story 8 = Process (framework scratch)

Key: "I didn't pitch framework — migrated 2 real, showed numbers, teams came to me. Pull, not push."

When to use which

Small ownership → Story 1 (12%→0.3%). Platform-lead → this Lazada variant. Same STAR timing.

PROBING — IF ASKED DEEPER

Why contracts not registry?

Registry = docs. Contract = enforcement. Drift back in sprint without it. Grace made it land.

Handle pushback?

RFC with 3 incidents + week-per-analysis cost + shadow pilot on my pipeline first. Data > opinion.

Streaming vs batch?

By consumer: sub-min → streaming, nightly → batch. Not one size.

08 // UNIVERSAL RULES

Universal Rules For Every Story

Apple rule: simple, visible, no clutter — same as your slides. Measurable + reflective + I not we. Every story ends with what you'd change.

I WHAT TO DO

I — Lead with I, follow with specifics
"I designed schema" not "we worked on it." Name trade-off, name scope, name decision.

⏱ **≤5 min, target 3:00–4:30**
4–5 questions in 30 min. Overrun costs volume. Time it aloud, record on phone.

% **Measurable outcome: %, $, hrs, latency**
12%→0.3%, 2wk→2d, 70%↓ pages, p95 3s→300ms. If can't quantify, say how you'd measure now.

End with what you'd change — growth signal
Rehearse closing line until natural, not rehearsed. Reflection = ownership.

! WHAT TO AVOID

⚖ **No heroes vs villains — trade-offs**
Frame conflict as engineering trade-off, not personalities. Learning > blame.

Silent prep fails — rehearse aloud
Record, listen for we vs I and filler. Mumbles hide in silent prep, not timed aloud.

? **Practice drill-down: "What if manager said no?"**
Follow-ups are where ownership is scored. Have partner ask hard follow-up per story.

Own narrative: S=business, T=your DoD
S=business + stakeholders, T=your definition of done, A=milestones+trade-off+comms, R=measurable+reflective.

Interview formula: Situation = business problem + why mattered. Task = your scope + done. Action = I milestones. Result = numbers + change.

Visible keys = what interviewer scores, not slides you show

09 // PRACTICE PROTOCOL — SHIP IT

Own The Narrative. Ship It.

Story 1: Led End-to-End (12%→0.3%) | Story 2: Disagreed Phased (5w 0 downtime) | Story 3: Framework (2wk→2d) | Story 4: Learned dbt+Snowflake | Story 5: Learned Go (1/10th mem, −40% cost)

01

Tailor to your real history

Replace samples with your GitHub. Real schema, real DAGs, real numbers. No fabrication — they probe.

02

Time aloud: 3:00–4:30

Not 5:00. Record on phone. Overrun costs you question volume. Aim 3:30 sweet spot.

03

Listen for we vs I

Count "we did" without your part. Rewrite to "I designed / I drove / I wrote / I profiled 12 tables".

04

Drill 1 follow-up per story

"What if manager said no?" "Why framework not template?" Follow-ups = where ownership is scored.

05

Rehearse closing line

"One thing I'd change..." until natural, not rehearsed. Growth signal. Natural > perfect.

06

Day before: 4 stories back-to-back

Under 20 min total. No notes. Final rep. Then stop — over-rehearsal kills presence.

✓ **Apple rule: simple, visible, no clutter — same as your slides. S = business + stakeholders, not tech first.**

5-MINUTE CAP MATTERS. GOING OVER COSTS YOU QUESTION VOLUME.